P0020r3 : Floating Point Atomic

Project:	ISO JTC1/SC22/WG21: Programming Language C++
Number:	P0020r3
Date:	2016-10-14
Reply-to:	hcedwar@sandia.gov
Author:	H. Carter Edwards
Contact:	hcedwar@sandia.gov
Author:	Hans Boehm
Contact:	hboehm@google.com
Author:	Olivier Giroux
Contact:	ogiroux@nvidia.com
Author:	JF Bastien
Contact:	jfbastien@apple.com
Author:	James Reus
Contact:	reus1@llnl.gov
Audience:	SG1 Concurrency, Library Evolution
URL:	https://github.com/kokkos/ISO-CPP-Papers/blob/master/P0020.rst

Revision History

P0020r3

- Align proposal with content of corresponding sections in N5131, 2016-07-15.

Overview / Motivation

This paper proposes an extension to the atomic operations library [atomics] for atomic addition on an object conforming to the atomic<T> where T is a *floating-point* type (N5131 3.9.1p8).

This paper **does not** include proposed extension of the named atomic type (N5131 Tables 134 and 135), or by implication extension of the C _Atomic qualification for floating point types.

The capability for atomic addition on floating point types critical for high performance computing (HPC) applications. The need is for extension of *atomic* and *atomic_view* (P0019) for floating point types.

Proposal

add to 29.2 Header <atomic> synopsis

namespace std {
  template<> struct atomic< *floating-point* >;

  // In the following declarations, *atomic-floating* is atomic<T>.

  floating-point atomic_fetch_add(volatile *atomic-floating-point**, *floating-point*) noexcept ;
  floating-point atomic_fetch_add(*atomic-floating-point**, *floating-point*) noexcept ;
  floating-point atomic_fetch_add_explicit(volatile *atomic-floating-point**, *floating-point* , memory_order) noexcept ;
  floating-point atomic_fetch_add_explicit(*atomic-floating-point**, *floating-point* , memory_order) noexcept ;
  floating-point atomic_fetch_sub(volatile *atomic-floating-point**, *floating-point*) noexcept ;
  floating-point atomic_fetch_sub(*atomic-floating-point**, *floating-point*) noexcept ;
  floating-point atomic_fetch_sub_explicit(volatile *atomic-floating-point**, *floating-point* , memory_order) noexcept ;
  floating-point atomic_fetch_sub_explicit(*atomic-floating-point**, *floating-point* , memory_order) noexcept ;

}

add to 29.5 Atomic Types

template<> struct atomic< *floating-point* > {
static constexpr bool is_always_lock_free = *implementation-defined* ;
bool is_lock_free() const volatile noexcept;
bool is_lock_free() const noexcept;
void store(*floating-point* , memory_order = memory_order_seq_cst) volatile noexcept;
void store(*floating-point* , memory_order = memory_order_seq_cst) noexcept;
floating-point load(memory_order = memory_order_seq_cst) volatile noexcept;
floating-point load(memory_order = memory_order_seq_cst) noexcept;
operator *floating-point* () volatile noexcept ;
operator *floating-point* () noexcept ;
floating-point exchange(*floating-point* , memory_order = memory_order_seq_cst) volatile noexcept;
floating-point exchange(*floating-point* , memory_order = memory_order_seq_cst) noexcept;
bool compare_exchange_weak(*floating-point* & , *floating-point* , memory_order , memory_order) volatile noexcept;
bool compare_exchange_weak(*floating-point* & , *floating-point* , memory_order , memory_order) noexcept;
bool compare_exchange_strong(*floating-point* & , *floating-point* , memory_order , memory_order) volatile noexcept;
bool compare_exchange_strong(*floating-point* & , *floating-point* , memory_order , memory_order) noexcept;
bool compare_exchange_weak(*floating-point* & , *floating-point* , memory_order = memory_order_seq_cst) volatile noexcept;
bool compare_exchange_weak(*floating-point* & , *floating-point* , memory_order = memory_order_seq_cst) noexcept;
bool compare_exchange_strong(*floating-point* &, *floating-point* , memory_order = memory_order_seq_cst) volatile noexcept;
bool compare_exchange_strong(*floating-point* &, *floating-point* , memory_order = memory_order_seq_cst) noexcept;

floating-point fetch_add(*floating-point* , memory_order = memory_order_seq_cst) volatile noexcept;
floating-point fetch_add(*floating-point* , memory_order = memory_order_seq_cst) noexcept;
floating-point fetch_sub(*floating-point* , memory_order = memory_order_seq_cst) volatile noexcept;
floating-point fetch_sub(*floating-point* , memory_order = memory_order_seq_cst) noexcept;

atomic() noexcept = default ;
constexpr atomic(*floating-point*) noexcept ;
atomic(const atomic &) = delete ;
atomic & operator = (const atomic &) = delete ;
atomic & operator = (const atomic &) volatile = delete ;
floating-point operator=(*floating-point*) volatile noexcept ;
floating-point operator=(*floating-point*) noexcept ;

floating-point operator+=(*floating-point*) volatile noexcept;
floating-point operator+=(*floating-point*) noexcept;
floating-point operator-=(*floating-point*) volatile noexcept;
floating-point operator-=(*floating-point*) noexcept;
};

add to 29.6.3 Arithmetic operations on atomic types

In the declarations of these functions and function template specializations, the name *floating-point* refers to a floating point type and the name *atomic-floating-point* refers to **atomic**<*floating-point*>.

in 29.6.5 Requirements for operations on atomic types

regarding arithmetic operations

C A::fetch_*key***(M operand, memory_order order = memory_order_seq_cst) volatile noexcept;**
C A::fetch_*key***(M operand, memory_order order = memory_order_seq_cst) noexcept;**

update 29.6.5p31 Remark

Remark: For signed integer types, arithmetic is defined to use two's complement representation ~~. There~~ and there are

no undefined results. For floating point types, if the result is not mathematically defined or not in the range of representable values for its type (5p4) the result is unspecified. [Note: Atomic arithmetic operations on *floating-point* should conform to **std::numeric_limits**< *floating-point* > traits associated with the floating point type (18.3.2). The floating point environment (26.4) for atomic arithmetic operations on *floating-point* may be different than the calling thread's floating point environment. - end note] For address types, the result may be an undefined address, but the operations otherwise have no undefined behavior.